

本项目分三部分，2.0为conformerASR语音识别模型的代码复现，3.0为conformer+squeezeformer的改良方案，_mindspore为mindspore架构上的3.0。

项目结构：

```
1 conformer_XXX（各版本结构相似） /
2 |
3 |— conf/                                # 配置文件
4 |   |— config.yaml                      # 模型与训练配置
5 |   |
6 |   |— data/                            # 数据处理脚本
7 |       |— aishell_data_process.py      # 生成数据清单
8 |       |— extract_data.py              # 解压数据集
9 |       |— tokenizer.py                 # 生成词汇表
10 |   |
11 |   |— model/                          # 模型代码
12 |       |— __init__.py                  # 模型入口（ConformerASR）
13 |       |— attention.py                 # 相对位置注意力机制
14 |       |— conformer.py                 # Conformer/Squeezeformer编码器
15 |       |— convolution.py               # 卷积模块
16 |       |— decoder.py                   # Transformer解码器
17 |       |— feed_forward.py              # 前馈网络
18 |   |
19 |   |— utils/                          # 工具模块
20 |       |— __init__.py                  # 工具入口
21 |       |— checkpoint.py                # 模型保存与加载
22 |       |— dataset.py                   # 数据集加载器
23 |       |— logger.py                    # 日志记录
24 |       |— metrics.py                   # 评估指标
25 |   |
26 |   |— data_aishell/                    # AIShell数据集（需自行准备）
27 |       |— wav/                         # 音频文件
28 |       |— transcript/                  # 标注文件
29 |   |
30 |   |— checkpoints/                     # 模型保存目录（自动创建）
31 |   |— logs/                            # 日志目录（自动创建）
32 |   |
33 |   |— train.py                          # 训练脚本
34 |   |— inference.py                     # 推理脚本
35 |   |— test_dataset.py                  # 测试集评估
36 |   |— test_train_set.py                # 训练集评估
```

数据集处理

本实验使用开源中文语音数据集 AISHELL-1，该数据集包含约178小时的中文录音，涵盖400位不同说话人的语音数据。数据预处理流程如下：

1.音频预处理:

首先读取WAV格式音频文件，验证文件完整性，过滤损坏或异常样本。然后将所有音频重采样至16kHz采样率，确保频率一致性。将多声道音频转换为单声道，采用平均混音策略。最后静音切除，基于能量阈值检测并移除音频首尾的静音段，保留有效语音内容。

2.特征提取:

提取80维梅尔频率倒谱系数（MFCC）特征作为模型输入。首先使用时频变换将时域信号转换为频域表示，使用短时傅里叶变换（STFT），帧长25ms、帧移10ms，并采用汉明窗。随后应用Mel滤波器组将线性频率刻度映射至Mel刻度，设置80个Mel频带，频率范围为20Hz至8000Hz；接着进行动态范围压缩，应用对数压缩增强频谱细节；最后进行特征归一化，采用均值方差归一化消除说话人差异。

3.文本处理与词表构建:

采用字符级建模方式，将所有文本转换为字符序列，并构建包含约4,300个字符的词汇表。文本清洗包括移除所有空白字符（空格、制表符等）、统一全角和半角标点符号、过滤非中文字符（保留数字和基础标点）。词表构建包含4,000个常用汉字作为基础字符，并添加4个特殊标记：（填充标记，ID=0）、（序列起始标记，ID=1）、（序列结束标记，ID=2）、（未知字符标记，ID=3），总计词汇表大小为4,004个条目。文本数字化将文本序列转换为ID序列，格式为[`char1,char2,...,charN`],[`char1,char2,...,charN`]。

4.数据增强:

在训练过程中采用SpecAugment策略，对频谱特征进行时间与频率维度的随机掩蔽，提升模型鲁棒性。通过在时间轴上随机选择连续的时间帧，将其所有频率值置为零，模拟语音中的短暂停顿或部分音频丢失；在频率轴上随机选择连续的Mel频带，将其所有时间值置为零，模拟频率选择性失真或部分频率丢失，从而增强模型对语音时序变化的鲁棒性和对声学环境变化的适应性。

5.数据集划分:

按照官方标准分割，训练集120,098个样本用于模型训练，开发集14,326个样本用于超参数调优和早停判断，测试集7,176个样本用于最终性能评估。

6.批处理:

采用动态批处理解决变长序列问题，通过使用零值填充至批次内最大长度，按序列长度降序排列最小化填充开销，并过滤过长或过短的音频样本确保训练稳定性。

训练配置

本实验采用端到端的训练策略，在PyTorch框架下进行模型训练与评估，通过精心设计的训练配置确保模型高效收敛与稳定训练。

1.优化器设置

采用AdamW优化器进行参数更新，该优化器结合了Adam的自适应学习率和权重衰减正则化。学习率调度采用余弦退火策略，在训练过程中平滑降低学习率，确保模型在训练后期能够精细调整参数。权重衰减系数设为0.01，有效控制模型复杂度，防止过拟合。

2.损失函数

采用CTC-Attention混合损失函数，结合连接主义时序分类损失与交叉熵损失的优势。其中CTC和Attention损失按比例加权求和，既保证了训练的稳定性，又提升了序列建模的准确性。

3.训练策略

训练批大小设置为64，通过动态批处理策略处理变长序列。采用早停机制防止过拟合，当验证集损失连续8轮不再下降时提前终止训练。梯度裁剪阈值为1.0，避免训练过程中梯度爆炸问题。同时采用梯度累积技术，每2个批次更新一次参数，在有限显存条件下支持更大等效批大小。

4.评估指标

本实验采用字错误率（Character Error Rate, CER）作为主要评估指标。为公平评估模型对中文字符的识别能力，计算CER时忽略所有空格，专注于字符本身的识别准确度。评估过程中使用动态规划算法（Levenshtein距离）精确计算编辑距离，确保评估结果的准确性。

解码器设计

为提升识别鲁棒性，本实验实现了三种解码策略，支持灵活切换：

1.CTC解码：

CTC基于编码器输出直接进行帧级分类，计算效率高，适合实时识别场景。CTC允许模型直接输出变长文本序列，无需严格的对齐标签，核心机制包括空白符号处理，路径折叠和贪婪解码。首先向词汇表中引入空白符号（blank token），用于处理音频帧与字符之间的不对齐问题；模型输出的帧级预测通过折叠操作，合并重复字符并移除空白符号，最终生成字符序列；在推理阶段，采用贪心解码策略，选择每一帧概率最高的字符作为输出，通过后处理得到最终识别结果。

2.注意力（Attention）解码：

注意力解码基于"选择性聚焦"机制，让解码器在生成每个输出时，动态地关注输入序列中最重要的部分，而不是使用固定的编码表示。在解码过程中，模型以当前解码器状态作为查询向量，计算其与所有编码器输出键向量的相似度，并通过softmax归一化为注意力权重分布，进而加权聚合得到上下文向量，该向量与历史输出共同输入解码器，通过全连接层预测下一个字符的概率分布。相比CTC解码的条件独立性假设，注意力解码显式建模了输出序列的马尔可夫依赖性，通常能生成更连贯的文本，但需要逐字符自回归生成，计算复杂度较高。

3.混合解码：

利用CTC解码的高效性和对齐稳定性，结合注意力解码的序列建模能力和语言连贯性，在推理阶段实现更可靠的识别结果。结合CTC与注意力解码的优势，当CTC输出置信度较低时，自动切换到注意力解码进行重打分，提升识别准确率。本实验在训练阶段采用CTC-Attention混合训练策略，采用CTC和Attention混合的解码方式，按比例加权同时计算CTC和Attention解码器的损失。

4.模型评估流程

模型评估采用分层验证策略：首先在训练集上随机抽样验证，监控模型在已知数据上的表现，及时发现过拟合现象；然后在独立测试集上进行全面评估，计算整体CER及分布统计，获取模型的真实性能；最后进行错误分析，根据CER分布情况识别模型在不同难度样本上的表现差异，为后续优化提供依据。

5.训练监控与保存

训练过程中记录训练损失、验证损失及字错误率变化，实时监控模型收敛情况。采用检查点机制定期保存模型参数，包括最佳模型、最新模型及周期模型，支持训练中断后的恢复。所有训练日志、评估结果及模型权重均按规范存储，确保实验的可重复性和可追溯性。

运行方式

一，pytorch框架下运行注意事项（即前两版代码）

环境配置

conformer2.0, 3.0需要pytorch3.9以上环境，包括os pandas numpy torch等深度学习常用工具包和tarfile tqdm math torchaudio argparse等辅助工具包。

数据集准备

- (1) 下载aishell数据集并解压，如果还未解压可借助项目文件夹中的data/extract_data.py脚本解压。
- (2) 将aishell数据集里的dev,train,test文件夹放到项目文件夹的data_aishell文件夹下，或者进入conf/config.yaml，更改data部分的数据_path路径为自己的路径。
- (3) 如果data文件夹下没有dev, test, train的csv文件，需先运行data/aishell_data_process.py文件生成。

模型测试

- (1) 确保checkpoints文件夹下有best_model.pth和latest_model.pth的权重文件，这是已训练好的模型。
- (2) 测试单个音频文件：

运行inference.py: `python inference.py --wav "data_aishell/wav/test/S0764/BAC009S0764W0121.wav"`

直接在终端运行即可。如果aishell数据集不在项目文件夹内，需把双引号内的路径改为自己的路径，测试音频可随意替换。

- (3) 测试训练集：

运行test_train_set.py: `python test_train_set.py --checkpoint checkpoints/best_model.pth --sample_count 1000`

确保conf/config.yaml内data_path路径正确即可。

本命令只测试前一千条样本，全训练集共约12万条样本，可修改--sample_count 1000决定测试数目，不建议全部测试，最好量力而行。

- (4) 测试验证集：

运行test_dataset.py: `python test_dataset.py --checkpoint checkpoints/best_model.pth --sample_count 50`

确保conf/config.yaml内data_path路径正确即可。

本命令只测试前50条样本，全验证集约七千条样本，去掉--sample_count 50可全部测试。

二，mindspore框架下运行注意事项（即最后一版代码）

为了方便验证，我们提供了两种方案：本地GPU和华为云Ascend NPU环境运行：

环境配置：

A：本地GPU环境（NVIDIA显卡）

1.系统要求

- 操作系统：Ubuntu 18.04/20.04/22.04 或 Windows 10/11
- GPU：NVIDIA GPU（建议显存 ≥ 8GB）
- CUDA：11.1 / 11.6 / 12.1
- Python：3.8 - 3.11

2.安装Mindspore GPU版本

```
1 # 创建虚拟环境
2 conda create -n mindspore python=3.9
3 conda activate mindspore
4
5 # 安装MindSpore GPU版（以CUDA 11.6为例）
6 pip install mindspore==2.2.0 -i https://pypi.tuna.tsinghua.edu.cn/simple
7
8 # 验证安装
9 python -c "import mindspore; print(mindspore.__version__)"
10
11 # 安装依赖库
12 pip install numpy pandas pyyaml tqdm librosa scipy
```

3.修改运行配置

```
1 # 在 train.py, inference.py 等文件开头修改
2 from mindspore import context
3 context.set_context(mode=context.GRAPH_MODE, device_target="GPU")
```

B：华为云Ascend NPU环境（推荐）

1.环境要求：

- 华为云ModelArts平台
- 镜像：mindspore_2.7.0rc1-cann_8.2.rc1-py_3.11-euler_2.10.11-aarch64-snt9b
- 算力规格：Ascend NPU

2.创建开发环境

1. 登录华为云ModelArts控制台
2. 进入 **开发环境** -> **Notebook**
3. 创建Notebook实例：
 - 选择镜像：MindSpore 2.7.0
 - 选择规格：Ascend NPU
4. 启动JupyterLab

3.验证环境

在JupyterLab终端中执行：

```
1 python -c "import mindspore; print(mindspore.__version__)"
2 # 应输出：2.7.0rc1
```

4.安装额外依赖

```
1 pip install librosa pyyaml pandas tqdm
```

数据集准备：

如果在本地GPU运行那么就pytorch框架下步骤一致，将下载好的data_aishell放进对应文件夹然后运行python aishell_data_process.py即可

如果在华为云服务器上面运行那么有两种方案，这里推荐第二种：

1.在线下载：

```
1 cd /home/ma-user/work/conformer_mindspore
2 mkdir -p data_aishell
3 cd data_aishell
4
5 # 下载数据集（约15GB）
6 wget -c https://openslr.magicdatatech.com/resources/33/data_aishell.tgz
7
8 # 解压
9 tar -xzf data_aishell.tgz
```

2.本地上传：

1. 在本地下载数据集：<https://www.openslr.org/33/>
2. 打包上传至服务器或通过OBS中转（直接搜索OBS，在jupyterLab上传时直接有OBS选项，一定要记得创建新桶）

执行如下代码：

```
1 cd data
2 python extract_data.py(如果没解压就执行此句)
3 python aishell_data_process.py
4 python tokenizer.py
5 ls -la data/
6 # 应看到: train.csv, dev.csv, test.csv, vocabulary.txt
```

模型训练：

直接运行即可：

```
1 python train.py
```

模型测试：

与pytorch框架下模型测试相同

注意：如果环境配置出现问题大概率是没安装相关依赖